

Delete All Occurrences in a Doubly Linked List

Personal DSA Notes

These notes follow the supplied transcript: traverse the DLL, delete every node whose value equals the key, update the head when necessary, repair the prev/next links, and return the updated head.

■filecite■turn26file0■L80-L120■

1. Problem Statement

You are given the head of a doubly linked list and a key/target. Delete **all occurrences** of that key. After deletion, the remaining nodes must still form a valid doubly linked list, and the updated head must be returned. The transcript illustrates an example where repeated 10s are removed and $4 \leftrightarrow 6$ remains.

■filecite■turn26file0■L24-L66■

2. Main Intuition

We do not need an extra data structure. Simply traverse the list using a temporary pointer. For every node, check whether its value equals the target.

If it does not match: move to the next node.

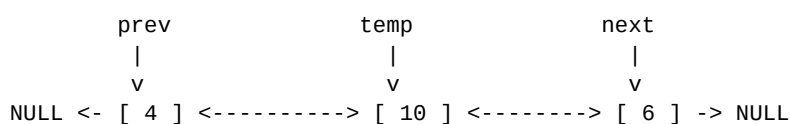
If it matches: delete that node by joining its previous and next nodes. If the node is the head, first move the head to head.next. Then continue from the next node.

The transcript emphasizes that the head itself should not be permanently used as the traversal pointer; the traversal uses a temporary pointer while the head is retained for the final return.

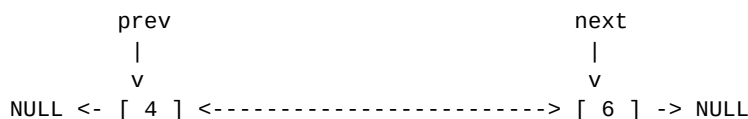
■filecite■turn26file0■L80-L94■

3. What Happens When We Delete a Node?

Before deletion:



After deleting temp:



Required link changes:

```
prev.next = next
next.prev = prev
```

Because this is a doubly linked list, both directions must be repaired when both neighboring nodes exist. The transcript explicitly describes these two possible link changes. ■filecite■turn26file0■L396-L430■

4. Special Case: Deleting the Head

Before:



```

[10] <-> [4] <-> [6]
prev      ^      ^
NULL      |      |
          ...    ...

```

Delete 10:

```
head = head.next
```

After:

```

HEAD
 |
 v
[4] <-> [6]
prev = NULL

```

When `temp == head`, the head must be updated. After saving the next node, that new head gets `prev = null` through the normal next-node link update. The transcript walks through this exact case.

■filecite■turn26file0■L94-L176■

5. Special Case: Deleting the Tail

Before:

```

[4] <-> [6] <-> [10] -> NULL
                        ^
                      temp

```

```

nextNode = null
prevNode = 6

```

There is no next node, so only:

```
prevNode.next = null
```

The transcript points out that when the deleted node has no next node, there is only one link change.

■filecite■turn26file0■L304-L342■

6. Why Save nextNode First?

List:

```

[4] <-> [10] <-> [10] <-> [6]
          ^
        temp

```

```

Save:
nextNode = temp.next

```

Delete first 10.

```

Then:
temp = nextNode

```

Now `temp` points to the second 10,
so consecutive target nodes are also deleted.

This is important because after removing the current node, traversal must continue with the node that originally came after it. The transcript repeatedly moves the temporary pointer to the previously saved next node. ■filecite■turn26file0■L178-L190■

7. Step-by-Step Algorithm

Step 1: Set temp = head.

Step 2: Traverse while temp is not null.

Step 3: If temp.val == target, start deletion.

Step 4: If temp is head, update head = head.next.

Step 5: Save nextNode = temp.next and prevNode = temp.prev.

Step 6: If nextNode exists, set nextNode.prev = prevNode.

Step 7: If prevNode exists, set prevNode.next = nextNode.

Step 8: Move temp = nextNode.

Step 9: If the value does not match, simply move temp = temp.next.

Step 10: Return head.

```
temp = head

while temp != null:

    if temp.val == target:

        if temp == head:
            head = head.next

        nextNode = temp.next
        prevNode = temp.prev

        if nextNode != null:
            nextNode.prev = prevNode

        if prevNode != null:
            prevNode.next = nextNode

        temp = nextNode

    else:
        temp = temp.next

return head
```

This follows the transcript's pseudocode structure and its warning that the null checks are necessary to avoid null-pointer exceptions. ■filecite■turn26file0■L350-L486■

8. Complete Java Code

```
class Solution {

    public ListNode deleteAllOccurrences(ListNode head, int target) {

        // Use temp for traversal so that head remains available
        // to return the updated linked list at the end.
        ListNode temp = head;

        while (temp != null) {

            // If the current node contains the target,
            // this node must be removed.
            if (temp.val == target) {

                // If we are deleting the head,
                // the next node becomes the new head.
                if (temp == head) {
                    head = head.next;
                }

                // Save both neighbours before changing links.
```

```

        ListNode nextNode = temp.next;
        ListNode prevNode = temp.prev;

        // If a next node exists, connect it backward
        // to the previous node.
        if (nextNode != null) {
            nextNode.prev = prevNode;
        }

        // If a previous node exists, connect it forward
        // to the next node.
        if (prevNode != null) {
            prevNode.next = nextNode;
        }

        // Continue from the node that followed
        // the deleted node.
        temp = nextNode;
    } else {

        // Current node is not the target,
        // so simply move forward.
        temp = temp.next;
    }
}

// Return the head of the updated doubly linked list.
return head;
}
}

```

9. Line-by-Line Code Understanding

temp = head; — starts traversal from the first node while preserving the head reference.

if (temp.val == target) — checks whether the current node must be deleted.

if (temp == head) — identifies deletion of the current head.

head = head.next; — moves the head to the next node.

nextNode = temp.next; — saves the node after the node being deleted.

prevNode = temp.prev; — saves the node before the node being deleted.

nextNode.prev = prevNode; — repairs the backward link when a next node exists.

prevNode.next = nextNode; — repairs the forward link when a previous node exists.

temp = nextNode; — continues traversal from the correct next node.

10. Why the Null Checks Are Mandatory

```

if (nextNode != null) {
    nextNode.prev = prevNode;
}

if (prevNode != null) {
    prevNode.next = nextNode;
}

```

At the head, prevNode is null, so prevNode.next cannot be accessed. At the tail, nextNode is null, so nextNode.prev cannot be accessed. Therefore both checks are required. The transcript specifically warns that removing these checks can cause a null-pointer exception. ■filecite■turn26file0■L464-L484■

11. Java Memory Note

The transcript mentions that in C++ the deleted node should be explicitly freed, while in Java garbage collection handles objects that are no longer reachable. ■filecite■turn26file0■L274-L292■

12. Edge Cases

- **Empty list:** temp is null, so nothing happens; return null.
- **One node = target:** head becomes null; return null.
- **One node ≠ target:** node remains unchanged.
- **Target at head:** head is updated to the next node.
- **Target in middle:** both neighbouring links are repaired.
- **Target at tail:** previous node's next becomes null.
- **Consecutive targets:** saved nextNode ensures every occurrence is checked.
- **All nodes = target:** every node is removed and the result is null.
- **Target absent:** traversal simply reaches null and the original head is returned.

13. Dry Run

Example:

```
10 <-> 4 <-> 10 <-> 10 <-> 6 <-> 10
target = 10
```

1. temp = first 10
It is head.
head = 4
4.prev = null
2. temp = 4
4 != 10
move forward.
3. temp = second 10
prev = 4, next = 10
4.next = next
next.prev = 4
move temp to saved next.
4. temp = third 10
Delete it in the same way.
5. temp = 6
6 != 10
move forward.
6. temp = last 10
prev = 6, next = null
6.next = null

Final:

```
4 <-> 6
```

This matches the transcript's traversal example, where repeated target nodes are removed and the remaining list becomes 4 ↔ 6. ■filecite■turn26file0■L292-L348■

14. Time Complexity — Clearly Explained

Time Complexity: O(n)

Let n be the number of nodes in the doubly linked list. We perform one traversal from the head toward null. Each node is visited once. When a node matches the target, the deletion itself uses only a constant number of pointer updates, so deleting a node costs $O(1)$. Therefore, across all n nodes, the total time is **$O(n)$** . This is the complexity stated in the transcript. ■filecite■turn26file0■L500-L512■

15. Space Complexity — Clearly Explained

Extra Space: $O(1)$

We do not create an array, HashMap, recursion stack, or another linked list. We only use a constant number of references: `temp`, `nextNode`, and `prevNode`. The amount of extra memory therefore does not grow with n , so the extra space is **$O(1)$** . The transcript states this as $O(1)$ space because only link changes are performed. ■filecite■turn26file0■L514-L520■

16. Complexity Summary

Time: $O(n)$
One traversal of n nodes.

Space: $O(1)$
Only a constant number of pointers are used.

This is optimal for this task because every node has to be checked at least once to know whether its value equals the target.

17. Quick Revision

DELETE ALL OCCURRENCES

```
temp = head
```

```
while temp != null:
```

```
    if temp.val == target:
```

```
        if temp == head:
            head = head.next
```

```
        nextNode = temp.next
        prevNode = temp.prev
```

```
        if nextNode != null:
            nextNode.prev = prevNode
```

```
        if prevNode != null:
            prevNode.next = nextNode
```

```
        temp = nextNode
```

```
    else:
        temp = temp.next
```

```
return head
```

Remember:

1. Save next.
2. Save prev.
3. Update head if needed.
4. Connect next $\leftrightarrow$ prev.
5. Move to saved next.
6. Return head.
7. $O(n)$ time, $O(1)$ extra space.